Technische Universität Berlin
Fakulät IV
Institut für Technische Informatik und Mikroelektronik

Learning and Intelligent systems Lab
Prof. Dr. Marc Toussaint

Internship Report

Plan-Based Reward-Shaping for Goal-Focused Exploration in Reinforcement Learning

Aditya Mohan

December 20, 2020

Contents

1 Introduction **6**

1.1 Motivation 6

1.2 Internship Objective 7

2 Theoretical Background **9**

2.1 Markov Decision Processes (MDP) 9

2.2 Value and Policy Iteration 10

2.3 State, Observation and Action Spaces 12

2.4 Q-Learning and Policy Gradient Methods 12

2.5 Reward Shaping 14

3 Method **15**

3.1 Experimental Set-up 15

3.1.1 Agent and Environment 15

3.1.2 RL Interface 16

3.1.3 Plan 16

3.1.4 Learning and Reward-shaping Methods 16

3.2 Training Pipeline 17

3.3 Testing Pipeline 17

3.4 Evaluation 18

4 Experiments **19**

4.1 Robotic Pushing Example 19

4.2 Pick and Place Example 21

4.3 Zig-Zag Example 23

5 Conclusion **25**

5.1 Future Outlook 27

List of Figures

1 Agent-Environment Model 9
2 Example MDP. Source:[23] 10
3 RAI Environment with the End-Effector 15
4 Training Pipeline 17
5 Testing Pipeline 18
6 Pushing Environment 19
7 generated Plan for Pushing Example 20
8 Evaluation of Pushing Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout 21
9 Pick and Place Environment 21
10 Pick and Place Plan 22
11 Evaluation of Pick-and-Place Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout 23
12 Comparison between the plan used for training the agent with 11 strides (Left) and the plan learned by the agent (Right) 25
13 Evaluation of Zig-Zag Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout 25
14 A 2 agent set-up where each agent has a different goal and they must compete against each other to push the box 28

List of Tables

1 Attributes of the Pushing Example Environment 19
2 Action Sequence to Generate plan for the Pushing example 20
3 Attributes of the Pick and Place environment 22
4 Action Sequence for Pick and Place environment 22

Acknowledgement

I want to thank Ingmar Schubert, my supervisor, for guiding me through this internship. The help and guidance that he provided during the preliminary studies were conducive to understanding the background related to the work of this internship. His valuable feedback during my work helped me understand concepts and implementations better. I am also thankful to Dr. Marc Toussaint for offering me the opportunity to work with the Learning and Intelligent Systems group during this internship. Finally, I would like to thank EIT Digital Master School and TU-Berlin for providing me the opportunity to study in the courses in AI and Robotics that were pivotal in introducing me to Dr. Toussaint and his research group's work.

Aditya Mohan
Berlin
December, 2020

1 Introduction

1.1 Motivation

One way to look at the behavior of agents [29] is by looking at how they interact with their environment, and how this interaction allows them to behave differently over time through the process called learning. In this view, the behavior of the agent can be modeled in a closed-loop manner through a fundamental description of the action and sensation loop. The agent receives input - sensation - from the environment through sensors, acts on the environment through actuators, and observes the impact of its action on its understanding of the environment through a mechanism of quantification in the form of the rewards it receives for its action. Reinforcement Learning (RL) is a paradigm in Machine Learning that uses this model of the agent-environment interaction to capture the important aspects of the problem facing a learning agent interacting with its environment to achieve a goal. It allows the agent to learn from the consequences of its actions, rather than from being explicitly taught. The methods in Reinforcement Learning are closed-loop because the learning system's actions can influence its later inputs, by affecting not only the immediate reward but also the next situation and, consequently, subsequent rewards. Being closed-loop, having no explicit instructions on the actions to take, and the play of the action-reward loop are what distinguish RL from Supervised or Unsupervised Learning. [1]

The three recurring paradigms in RL can be categorized as follows [11]:

1. **Learning v/s Planning:** In learning the rules of the game are unknown and are learned by putting the agent in the environment, while planning is driven by a model of the rules that need to be followed, which can be used by the agent to perform a look-ahead search on the actions that it can take.
2. **Exploration v/s Exploitation:** The fundamental choice that an agent has for any decision is to either exploit the knowledge it already has to take an action that maximizes the reward, or to forego this 'obvious' choice and explore the environment in order to discover better paths to the goal.
3. **Prediction and Control:** Prediction is finding a path to the goal, while control is optimizing this path to the goal.

The problem of exploration is particularly interesting because of the nature of the state-space in which the agent operates. In high-dimensional state-spaces, RL algorithms suffer

from the Hard-Exploration problem [9] due to sparse and deceptive rewards (E.g. the Noisy TV problem [7]) and the sheer number of possibilities of paths that can be explored. This makes learning extremely slow and sample-inefficient, especially considering its application to many real-world tasks.

1.2 Internship Objective

One way to increase the sample-efficiency is through Reward-Shaping. It is a technique inspired by animal training in which the reward to the agent modified in such a way so that the agent can guide it's exploration towards a desired direction or Path [28]. For example, in a sparse reward setting where the agent receives a reward of -1 for each action that leads to a position not in a certain vicinity of the goal and 0 for reaching the goal, shaping the reward by inculcating the agent's distance to the goal helps the agent learn to reach the goal much quicker [26].

The essential question, then, becomes whether this reward transformation preserves the optimality of the policy that the agent learns i.e is the agent still learning the same problem, or has the problem inherently been transformed. A specific kind of condition that has been introduced in [19] is that of preserving this optimality through reward functions that are of the same nature as potential functions in a field. This is called Potential-Based Reward Shaping (PBRS).

Another approach to reward shaping is by using it as a method to inculcate priors related to the environment into the agent's exploration. For example, In the case of the agent's position w.r.t the goal being used in reward shaping, another prior that can be used is a plan of actions that can lead to the reward. So, the shaping is done by seeing how close a path learned by the agent is to this plan. In the domain of Robotics, approximate domain knowledge can usually be used by a planning algorithm to generate plans that an agent might follow. Thus, inculcating these plans through reward shaping is called Plan-Based Reward Shaping [10].

The objective of this internship is to look into one such method of Plan-Based Reward shaping, proposed in [2], that introduces a new class of Plan-Based Reward shaping that relaxes the optimality guarantee imposed by PBRS and accelerates learning by generating policies that only lead to the same final state as opposed to preserving optimality. This method is called Final-Volume-Preserving Reward Shaping (FVRS). The specific tasks that were done by me in this internship was to look how this approach performs in different

kinds of scenarios and to compare them based on certain metrics. This involved setting-up a Deep Learning Training, Testing and Evaluation pipeline on a High-Performance Cluster and effectively analyzing the results.

2 Theoretical Background

2.1 Markov Decision Processes (MDP)

The Reinforcement Learning Model introduced in the previous section is summarized in Figure 1.

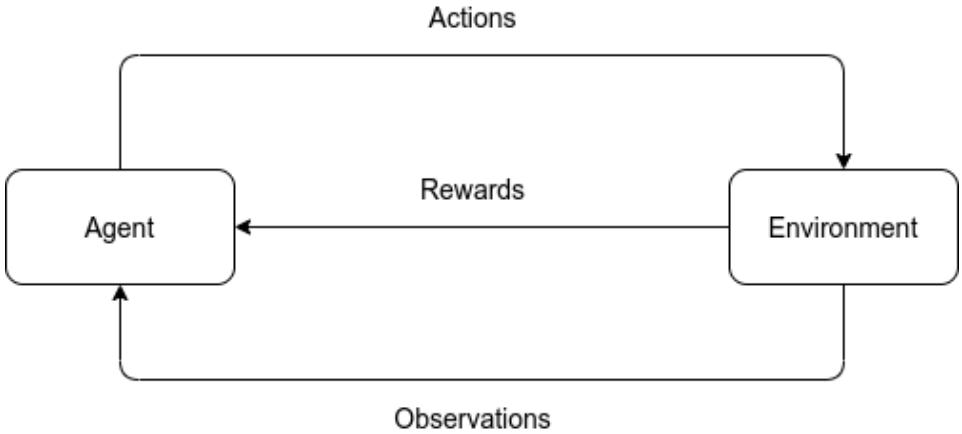

Figure 1: Agent-Environment Model

The Box pertaining to the Environment can also be interpreted either as the world (when the agent has the ability to observe the world fully), or as the agent's understanding of the world (In the case when the agent has a limited partial-observability of the world). the world has a lot of attributes that might influence, and be influenced, by the agent's action. Thus, a method to model these attributes is highly useful since it allows the agent's behaviour to be interpreted in the form of transitions. This can be further formalized by defining the observations as a certain 'State' in which the agent might exist. The state might either be equal to the observation, or be a subset of the observation, depending on the problem.

Markov processes are random processes indexed by time and are used to model systems that have limited memory of the past [15]. The fundamental intuition behind Markov processes is the property that the future is independent of the past, given the present. Thus, the present state of the agent can be conditioned on a limited number of previous states, and not the whole history of its states or actions. This 'window' over which the state is conditioned determines the order of the Markov Process. For first-order processes, the state only depends on the previous state, and so the conditional equation can be written as shown in equation 1

$$P(S_{t+1}|S_t) = P(S_{t+1}|S_1, ..., S_t) \tag{1}$$

Using this property - called the Markov Property - the Environment can be modeled as a Directed graph in which the nodes represent the states in which an agent can exist and the edges represent the transitions in these states. When for each state, there are multiple transitions possible, the agent has to decide as to which transitions to follow, and this is called a Markov Decision Process (MDP) [5]. An example of such a process is shown in Figure 2

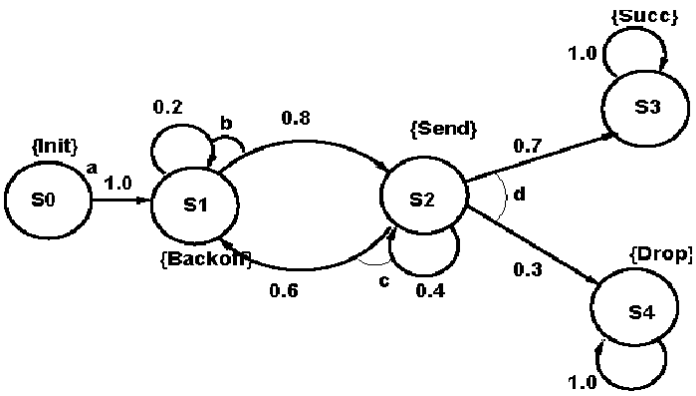

Figure 2: Example MDP. Source:[23]

Now, the RL problem can be reformulated as a solution to this MDP by formulating a mechanism to maximize the reward.

2.2 Value and Policy Iteration

The MDP can be formulated as $< S, A, T, \gamma, R >$ where,

- **S** : Set of all possible states.
- **A** : Set of all possible actions.
- **T** : Transition Dynamics of the system where $T(s'|s, a)$ is the probability density of the next state being s'.
- **R** : Reward for each transition.
- γ : the factor that discounts future rewards.

The schema that the agent follows to make its decisions in an MDP to accomplish its goal is called a policy. Thus, a policy is a mapping from states to actions and can be understood as the probability of taking a certain action in a given state, formulated as shown in Equation 2.

$$\pi(a|s) = P(A = a_t|S = s_t) \tag{2}$$

The discounted return of a rollout can be defined as shown in Equation 3. This is a measure the rewards accumulated from the current state into the future. The value of γ controls the shadow of the future that the agent uses to measure the impact of future rewards.

$$G_t = \sum_{i=1}^{\infty} \gamma^{i-1} R_{t+i} \tag{3}$$

The value of a state and action under any policy π can be written as shown in Equations 4.

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t|s_t = s] \\ q_\pi(s,a) &= \mathbb{E}_\pi[G_t|s_t = s, a_t = a] \end{aligned} \tag{4}$$

The goal of the agent is to learn an optimal policy π^* that maximizes the expected discounted return G_t defined in Equation 5. This means that when the agent operates under the policy π^*, then it selects the next states in a way so as to maximize the expectation of the discounted rewards from that state, into the future.

$$\pi^* = \arg\max_\pi \mathbb{E}_\pi[G_t] \tag{5}$$

The agent does this either by iterating over the value of the states and actions - Value Iteration - or by directly iterating over the learned policies - Policy Iteration. In value iteration, the formulation of the optimal policy can be made recursively using the Bellman Update [1], as shown in Equation 6

$$\begin{aligned} v^*(s) &= \max_a \mathbb{E}[r + \gamma v^*(s_{t+1})|s_t = s] \\ q_\pi(s,a) &= \mathbb{E}[r + \gamma \max_{a_{t+1}} q^*(s_{t+1}, a_{t+1})|s_t = s, a_t = a] \end{aligned} \tag{6}$$

Policy iteration, on the other hand, first extracts the policy from the current value function through a step called policy extraction and then runs value iteration conditioned on this policy through policy evaluation.

2.3 State, Observation and Action Spaces

If the world that the agent interprets in its action-environment interface shown in Figure 1 is fully observable, then the state of the agent is identical to its observation. However, if this is not the case then the observation is mapped as a belief over possible states, and in this case, the MDP formulated is called a Partially-Observable MDP, or POMDP.

Observation spaces can be discretely or continuously valued. An example of a discrete observation space is the layout of a tic-tac-toe board, where empty space is represented by a zero, and pieces can be represented by the numbers 1 and 2 for x and 0, respectively [24]. A continuous observation space example might be a vector of joint torques and velocities in a robot [3].

Actions can be continuously or discretely valued. An example of a discrete action is one in Atari where the action space is characterized by four distinct directions of control for the Agent [4]. Continuous actions are more complex. One example of this would be a real-valued vector of continuous numbers representing amounts of force to apply to the robot's joints.

2.4 Q-Learning and Policy Gradient Methods

Q-Learning [27] is a model-free method that directly approximates the optimal action-value function q, independent of the current policy. It has 3 steps that are repeatedly applied for convergence:

1. Start in state s_t and pick an action according to the policy defined by the q function.
2. Take the action and gather the rewards
3. Apply the update rule given in Equation 7

$$q(s_t, a_t) \longleftarrow q(s_t, a_t) + \alpha[r_t + \gamma \max_{a_{t+1}} q(s_{t+1}, a_{t+1}) - q(s_t, a_t)] \tag{7}$$

The vanilla way would be tabular Q-Learning where the q function is represented as a table of a designated number of actions for each state. Thus, the algorithm would be selecting the action with the maximum q-value for the visited state and updating the q-values according to Equation 7. This method, however, becomes intractable for higher dimensional state ad action spaces. To tackle this, [17] and [18] introduce the Deep Q-learning, which uses a deep neural network as the Q function approximator and applies gradient descent to minimize the

objective function which is through this network.

The major issue with DQN, and by extension Q-Learning, is that each possible action is assigned a score that is used to evaluate it in the future. This methodology breaks down for continuous action spaces since the only way to apply this class of techniques to continuous action values is to discretize the space and this added limits and complexity to the task. Now, if the state space is also continuous, then this becomes even more complicated. One workaround would be the discretization of these spaces, but that would bring the added problem of preserving the information effectively.

Policy Gradient Methods [22] are alternate approaches to Q-Learning which work by directly optimizing in the policy space. The basic idea is to use a neural network (or other function approximators) to directly model the action probabilities. Each time the agent interacts with the environment it generates a data point. This is used to tweak the parameters of the neural network so that favourable actions become more likely to be sampled in the future. This process is repeated until the policy network converges to the optimal policy.

Policy Gradient methods have the advantage of allowing the agent to have an 'understanding' of the maximum instead of directly computing it as is the case with Q-Learning. Thus, they are better suited for continuous spaces. These methods, however, suffer from the disadvantages of tending to converge to local optima and they take much longer to train.

Actor-Critic methods [12] overcome these problems by creating a hybrid architecture of value and policy iteration. These methods use two Neural networks:

- **Actor :** A policy-based method to generate action to control the agent behaves.
- **Critic :** A value-based method to measure the quality of the action

Deep Deterministic Policy Gradients (DDPG) [13] is an actor-Critic method that leverages the advantages of DQN for its Actor Network and uses Policy Gradients for its critic network. For the Q-Learning part, it uses a replay buffer to store the past experiences and defines a time-lagged target network to use for comparing the fit of the Q-Value to the Bellman Equation. This is done for both actor and critic networks. To evaluate the actions it uses a Target policy network that maximizes the target Q-Value. The policy iteration is simply done by treating the Q-function as differentiable over the action-space and maximizing this gradient using gradient-ascent.

The exploration of the DDPG policies is controlled by either adding time-correlated Noise or Zero-Mean Gaussian Noise.

2.5 Reward Shaping

As mentioned in Section 1, the problem with sparse rewards is that the exploration becomes hard and sample inefficient. One way to alleviate this problem is through Reward Shaping, where the reward received by the agent is modified from that of the signal given by the original MDP to one that guides this exploration better. This can be formulated by adding a shaping function, F, to the original reward function R as shown in equation 8.

$$R'(s, a, s') = R(s, a, s') + F(s, a, s') \tag{8}$$

The optimal policy π'^* learned by using R' as a reward signal, however, would be different from the optimal policy π^* of the original MDP. To make these policies equal, [19] presents PBRS that constraints F to be a potential function of the form shown in equation 9.

$$F(s, a, s') = \gamma\phi(s') - \phi(s) \tag{9}$$

The work done in [2] introduces a notion of compatibility that is a super-set to the notion of policy invariance. To achieve this, it first establishes the notion of an optimal final volume of an MDP. This can be understood as the minimal set of states that an agent needs to visit in order to be able to re-construct the underlying MDP i.e account for all the dynamics of the underlying MDP. This notion becomes increasingly important for classes of problems where if such a set of states is found, then the probability of the agent leaving this volume is close to 0. It then takes the case of two MDPs with the same structure but different rewards and discounting factors, that generate different optimal policies, and defines them to be compatible if and only if after a finite amount of time, the state of one of these MDPs ends of being inside the optimal final volume of the other MDP. This means that if an agent behaves optimally in one MDP - say M' - and this MDP is compatible with an MDP - say M - then the agent ends up displaying behavior that will give a result that is asymptotically similar to the optimal behavior in M. The key finding here is that the result in M' can be obtained by shaping the reward in a way that increases the sample-efficiency.

Now, this behavior in M' can be generated by executing a plan generated by a planner that leads to the goal, and shaping the reward based on the closeness to this plan. This allows the agent to exploit the planner and inculcate it apriori to develop a goal-focused exploration strategy that generates the same behavior that the agent might learn with much more samples if it didn't have any reward shaping.

3 Method

This section details the method that was followed to generate plan-based exploration strategies as explained in the previous section.

3.1 Experimental Set-up

3.1.1 Agent and Environment

The agent used is an End-Effector that operates in a 3D space. A general rigid body has 6 degrees of freedom (3 positional coordinates, and 3 rotational). Since the finger is spherically symmetric, the positional coordinates suffice since all rotational states are equivalent. Hence, the 3 dimensions for the finger are sufficient to describe its movement. The object, is either a cuboid that does not have symmetric rotations, and so needs to be described by a 3D vector of translations and a 4D quarternion of rotations, or a disc that can be described by 3D position and 1D non-symmetric rotation about the z-axis. When these are concatenated, we get the state as a 10D vector.

The environment used is RAI [21], which is an open sourced-simulator developed using NVIDIA PhysX engine [20] to simulate a scenario where objects are kept on the table. A sample of this environment set-up is shown in figure 3

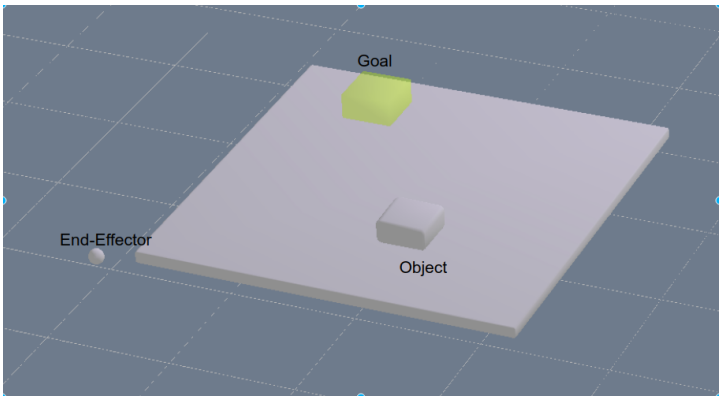

Figure 3: RAI Environment with the End-Effector

The RAI environment implements physics so that the actions that are performed by the agent follow physical laws similar to real-world objects, except for the residual forces that might exist in the real world.

3.1.2 RL Interface

The RL agent-environment interface is created through a wrapper around this environment, specific to each of the examples discussed in section 4. The common features shared by all these environments are as follows:

1. The action taken by the agent is simulated by its movement along the 3 axes and is fed into the environment in the form of a 3D vector of floating-point values
2. The observation generated by the environment is state of the object - position and rotation - to which the EE can move the object.
3. The reward given to the agent is 1 if and only if the object moves to distance 0.1 units from the goal position in any direction, and in all other places, the agent receives a reward of 0.

3.1.3 Plan

The direction of this work is to integrate a 'plan' to guide exploration. Ideally, this plan should be generated by the KOMO-Planner [25] of RAI. However, for this set-up, this was simulated by hand to generate a sequence of actions that are executed to generate a sequence of observations, which can be used as the plan. Thus, if the FVRS performs well in this environment, then it can be extended to the integration with KOMO too.

3.1.4 Learning and Reward-shaping Methods

The observation that is received after taking an action in the agent-environment interface is passed on a reward shaping unction that compares the euclidean distance of this observation to all the elements of the set of observations that constitute the plan and then uses a Gaussian function to generate a shaped reward. A Gaussian plane-based function is used for shaping the reward as shown in equation 10, where $\delta(s)$ is the distance between the observation of state s from a point in the plan and w is the length of this Gaussian window, which has been taken as 0.2. K(s) is the minimum distance of a observation to the generated plan

$$F(s) = \arg\min_{\delta(s)} 0.5 \frac{k(s)+1}{L} \exp\left(-\frac{\delta^2(s)}{2w^2}\right) \tag{10}$$

In this set-up, the actions and the observations are all continuous. Thus, the method that

is used for training is primarily DDPG, which is one of the main methods [2] uses for its experiments. The exploration acts as ϵ-greedy with a factor of 0.2. The Actor and Critic networks are implemented using Keras [8], on a TensorFlow backend [16].

3.2 Training Pipeline

The training was done for a maximum of 2000 roll-outs, each of a length uniformly sampled between 0 and 300, with the agent always starting from the same position. The pipeline is shown in figure 4.

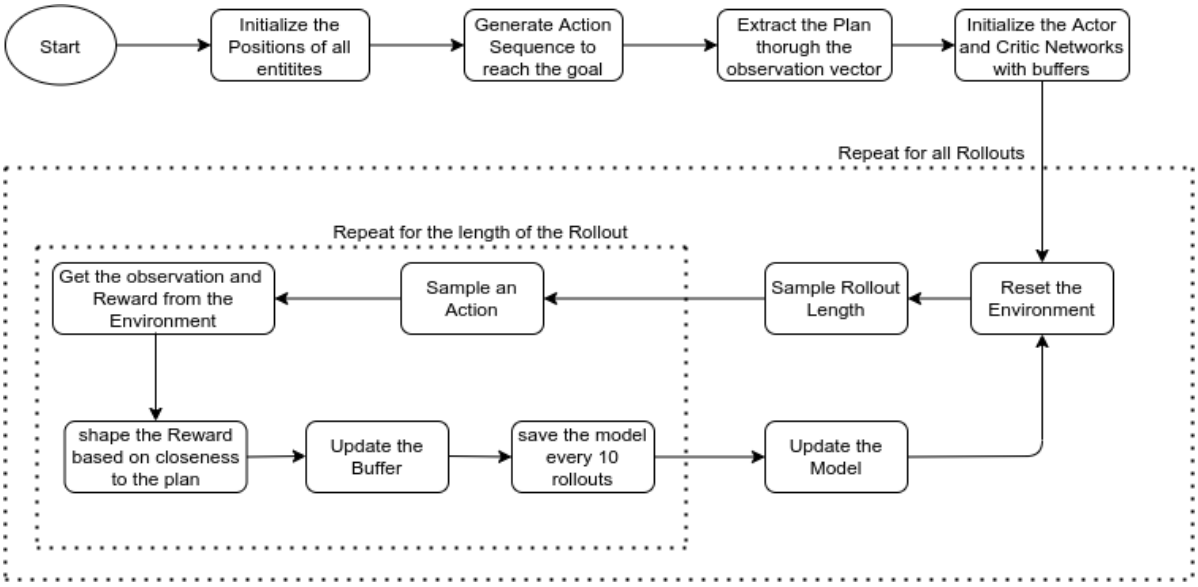

Figure 4: Training Pipeline

This pipeline is repeated for Multiple agents and the data is stored for all these agents in the form of pickle files.

3.3 Testing Pipeline

Once the agents have been trained, the testing is performed by executing their stored policies and measuring the final distances they reach w.r.t the goal position. This is done for multiple samples per rollout and the final distances are dumped in a pickle file. Figure 5.

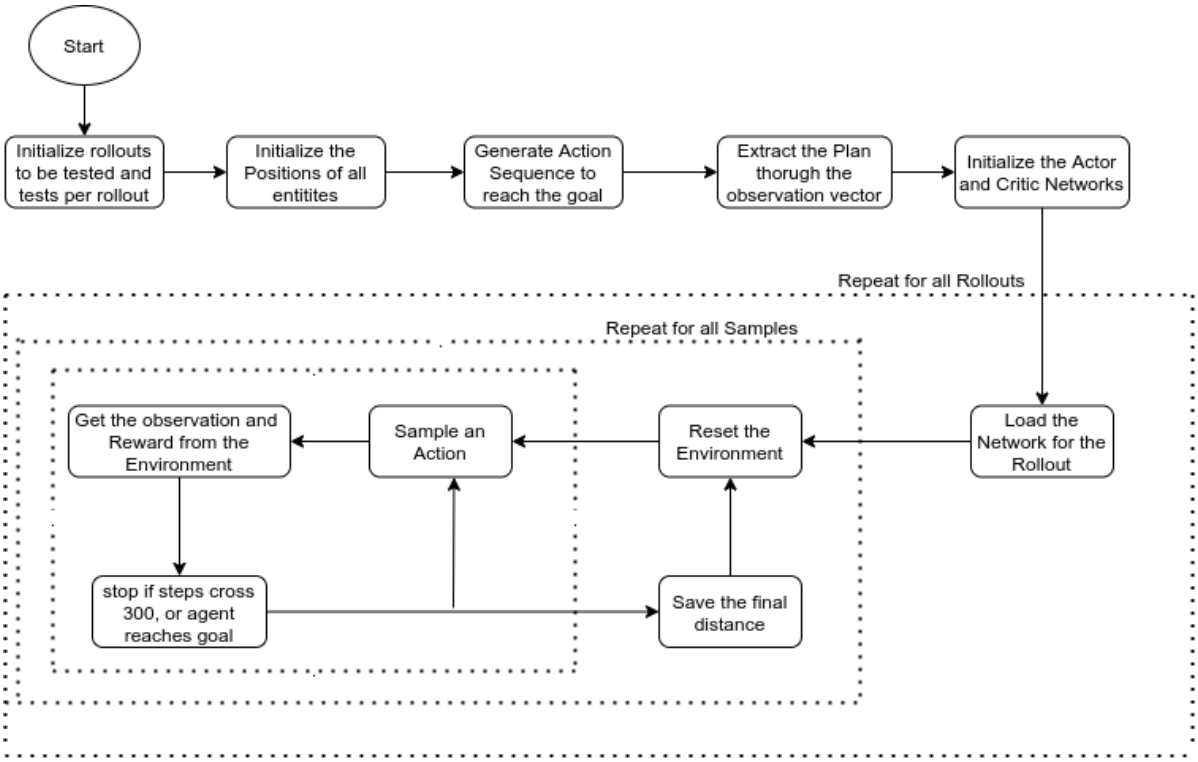

Figure 5: Testing Pipeline

3.4 Evaluation

Once the tests have been performed for all agents, the evaluation is performed by loading the final distances that have been stored and plotting the success rates for the agents. The following steps are followed for the evaluation:

1. Extract the data into a 3D vector of Agents x samples x Rollouts
2. Average the final distances of each agent through all the sample to get an Agent x Rollouts shaped array
3. For each rollout, count the number of agents that reached within 0.1 units of the goal
4. For each of the agents that reached the goal, log the number of rollouts after which these agents reached the goal for the first time
5. Log the mean distances for the agents that reached closest to the goal
6. Plot the average distances for all the agents and the ratio of agents that reached the goal

These steps generate plots that have been shown in the following section.

4 Experiments

This section lists the examples that were created to test-out the FVRS approach and the final results for each of these examples.

4.1 Robotic Pushing Example

In this example, the specification of the setup is shown in 6

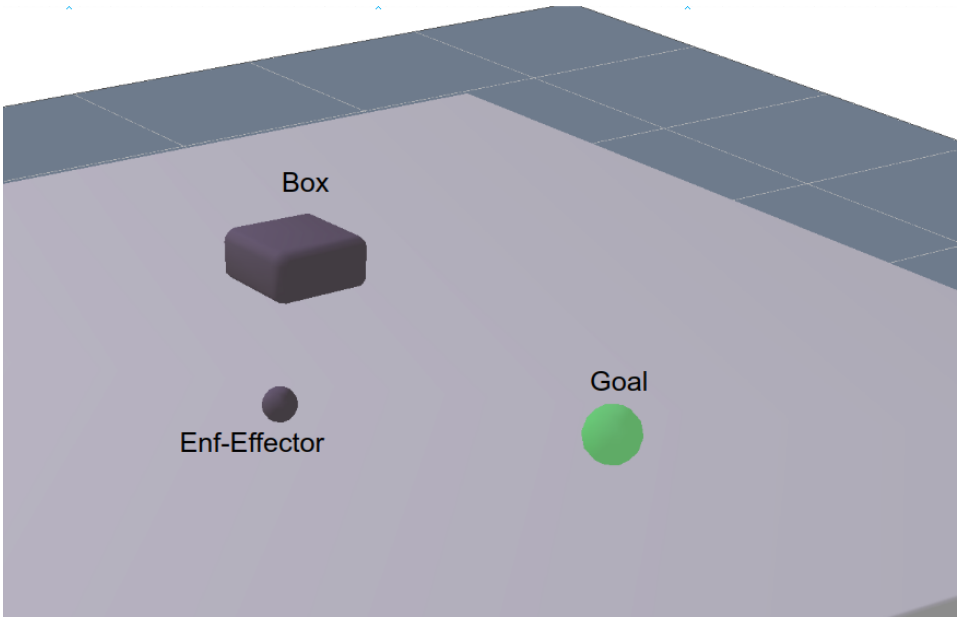

Figure 6: Pushing Environment

The attributes of this environment are listed in Table 1

The action sequence used to generate the plan is shown in Table 2

The generated plan is shown in figure 7.

Attribute	Value
Size of Box	$0.4 \times 0.4 \times 0.2$
Size of Table	3×3
End-Effector Initial Position	(-0.6, 0.5)
Box Initial Position	(-1.0, -1.0)
Goal Position	(0.0, 0.0)

Table 1: Attributes of the Pushing Example Environment

Action	Duration
(-0.05, 0, 0)	13
(0, -0.05, 0)	32
(0.05, 0, 0)	19
(0, -0.05, 0)	3
(0.05, 0, 0)	6
(0, 0.05, 0)	20

Table 2: Action Sequence to Generate plan for the Pushing example

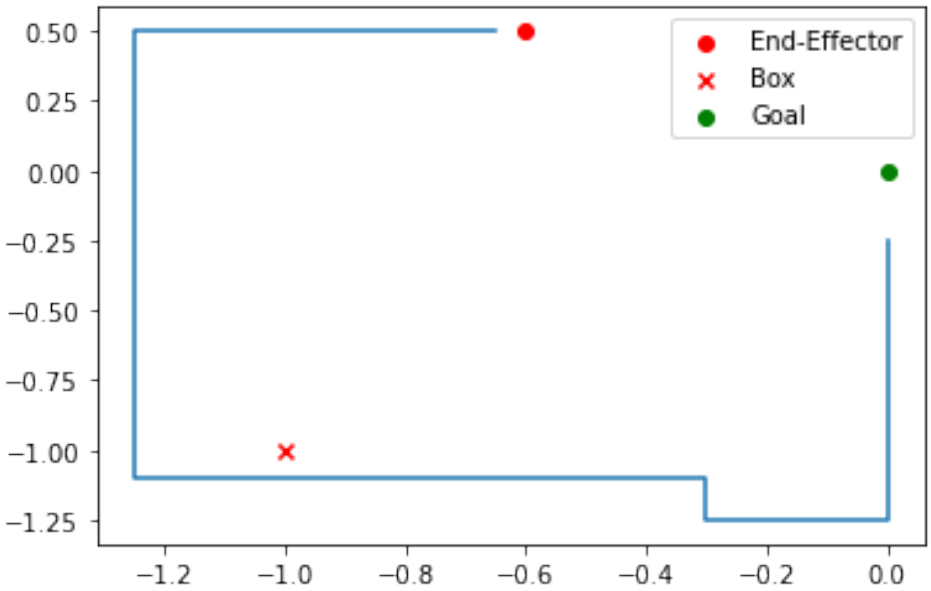

Figure 7: generated Plan for Pushing Example

For this example, 20 agents were trained for 2000 rollouts each and the evaluation was done for every 50th rollout, starting from rollout 1 and ending on rollout 1501. Through the evaluation methodology described in the previous section, the results obtained have been plotted in figure 8. The factor that shows the success of the FVRS approach is the decrease in the mean asymptotic distance to the goal as the rollouts progress, and it can be seen that close to 240 rollouts the agents on an average have already learned to push the box within 0.1 units of the goal, and this fraction keeps growing from 200 rollouts to the end of the process. Thus, we can say that in the long run, the agent learns to behave optimally in the task of reaching within 0.1 units of the goal successfully with just 200 rollouts, when the reward is shaped based on the closeness of the movement to the plan.

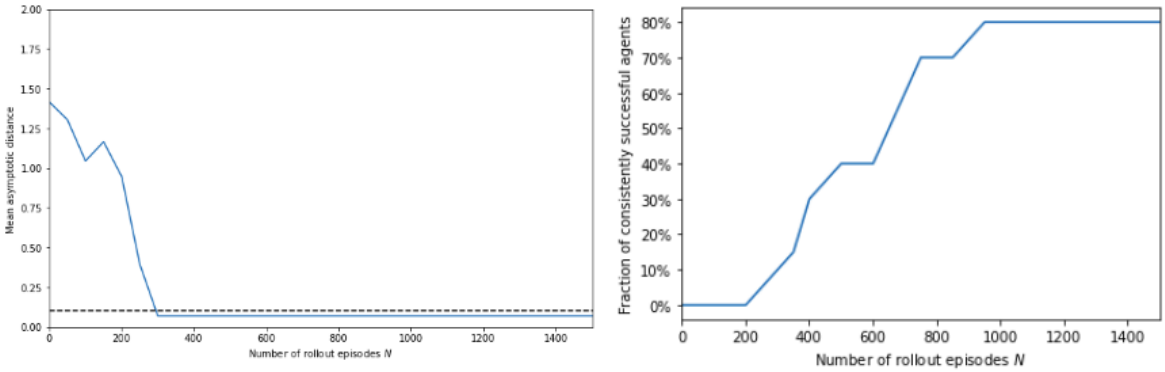

Figure 8: Evaluation of Pushing Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout

4.2 Pick and Place Example

The set-up of this environment is shown in Figure 9. In this environment the action of picking and placing has been implemented by checking for a minimum distance - contact distance - between the frames of the finger and the disk and once the finger is within this range, the frames are moved together for the distances calculated form the action sequence. In essence, this allows the render and the algorithm to emulate the action of picking and placing and object.

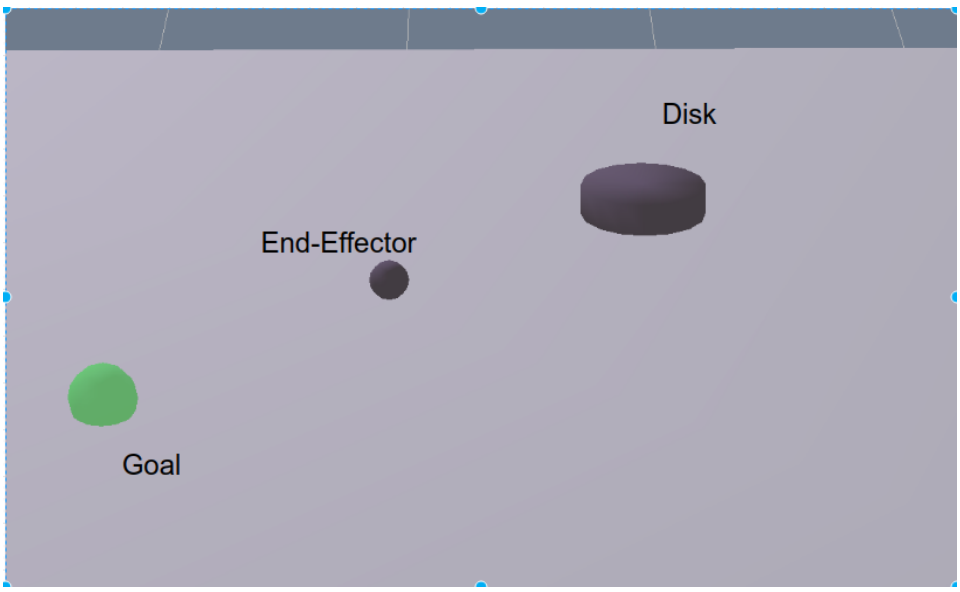

Figure 9: Pick and Place Environment

The attributes of this environment are listed in Table 3.

Attribute	Value
Radius of Disk	0.2
Size of Table	3 × 3
End-Effector Initial Position	(0, 0)
Box Initial Position	(-0.8, -0.8)
Goal Position	(0.8, 0.8)

Table 3: Attributes of the Pick and Place environment

Action	Duration
(0, 0, 0.05)	5
$\frac{1}{2\sqrt{2}}(-1,-1,0)$	22
(0, 0, -0.05)	4
(0, 0, 0.05)	4
$\frac{1}{2\sqrt{2}}(1,1,0)$	45
(0, 0, -0.05)	4

Table 4: Action Sequence for Pick and Place environment

The action sequence that was used in this environment as shown in Table 4.

The plan produced by this action sequence is shown in Figure 10.

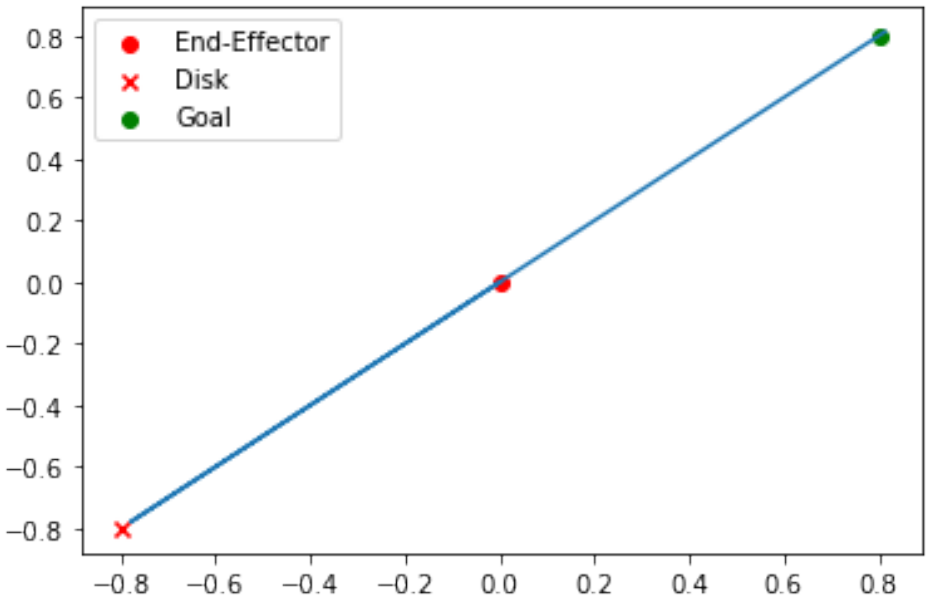

Figure 10: Pick and Place Plan

This example is also evaluated in a manner similar to the pushing example with the same metrics and the same asymptotic distance of 0.1 within the goal vicinity. The results have

been summarized in Figure 11. Here, similar behavior of the mean asymptotic distance as the pushing example, where the agents learn to place the disk in the 0.1 unit vicinity of the goal in around 250 rollouts, can be seen. The differentiating factor, however, is the fraction of consistently successful agents, which starts to rise much before the 200 mark that was seen in the pushing example. There are multiple explanations for this. In one view, this could be attributed to the topological nature of the plan that was initially generated since the plan for this example traversed along a straight line as compared to the plan in the pushing example. In other words, the rewards that the agent received in the initial stages was sufficiently effective at leading the agent to the goal as compared to the reward the agent received in the pushing task, due to the nature of the plan that was used. As the agent learns to go along the straight line at the start, that effectively guides it to the goal as compared to the turns that are seen in the pushing example. Consequently, more agents are successful earlier-on than the pushing example. The other viewpoint, which is albeit more simpler, is that pushing is inherently a more complex interaction than pick-and-place, especially in the manner that the latter has been implemented in this environment.

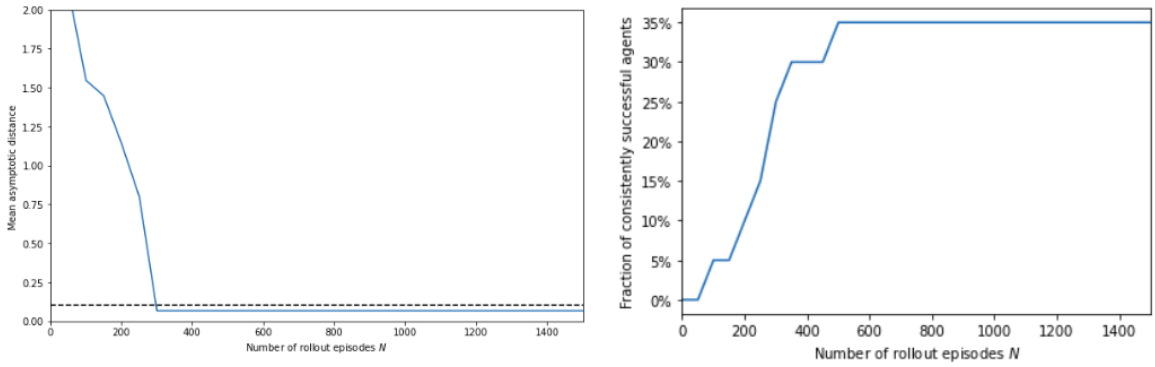

Figure 11: Evaluation of Pick-and-Place Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout

4.3 Zig-Zag Example

This environment is a modification of the Pick and Place environment where the plan is changed for the same EE, Disk, and Goal specification. The idea behind this example is to create a plan that has rich information that is densely packed in the euclidean space that can be exploited by the agent without any constraints to go to a goal. The question that arises, thus, is whether the agent learns a simpler path to the goal than the one proposed by the

plan? In other words, subject to the constraints imposed by the environment, is the agent able to figure out a 'better' solution than the one given by the plan?

The plan that has been created is a zig-zag path to the goal. This is a tunable path, where each movement along the zig-zag path is called a stride and the number of strides (N) determines the path taken by the agent. Thus the plan of action can be summarized as follows:

1. Go straight to the disk
2. Pick-up the disk.
3. Reach the goal by taking a pre-defined number of strides
4. Drop the disk

The first and second parts of the action sequence would be the same as the Pick-and-Place example. The third part is where the tunable nature of the plan comes into place. For 3 strides, the spacing between the strides is a lot and so the agent is expected to learn a path that might be close to the plan. The assertion is that as the strides increase, the information of the plan gets more densely packed while the 'usable' information for the agent does not really increase by a lot. Thus the agent is able to exploit its closeness to two strides that close together and figure out a connecting straight line towards the goal. Hence, in a finite number of strides, the agent should be able to learn the same path that it learned in the vanilla Pick-and-Place example.

Figure 12 shows the comparison between the plan that was fed for reward-shaping and the path that the agent learned, for N=11. It can be observed that the agent does in-fact learn a better solution than the one proposed by the pan and the path learned by the agent is very similar to the vanilla example.

The training of these agents followed the same process as the previous examples and the evaluation followed the same metrics. The results are shown in Figure 13. It can be seen that even though the agent learns a new path from that of the plan, the sample efficiency falls in the same range as that of the previous examples as seen by the mean-asymptotic distance coming in the vicinity of the goal in just 200 rollouts. Moreover, the agents learn this optimal path pretty quickly as seen in the fraction of consistently successful agents starting to rise around the 200 rollouts.

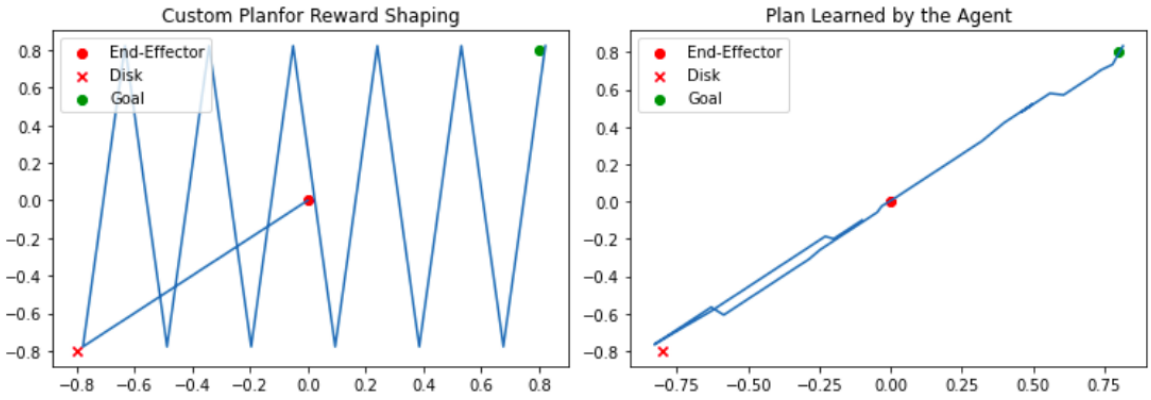

Figure 12: Comparison between the plan used for training the agent with 11 strides (Left) and the plan learned by the agent (Right)

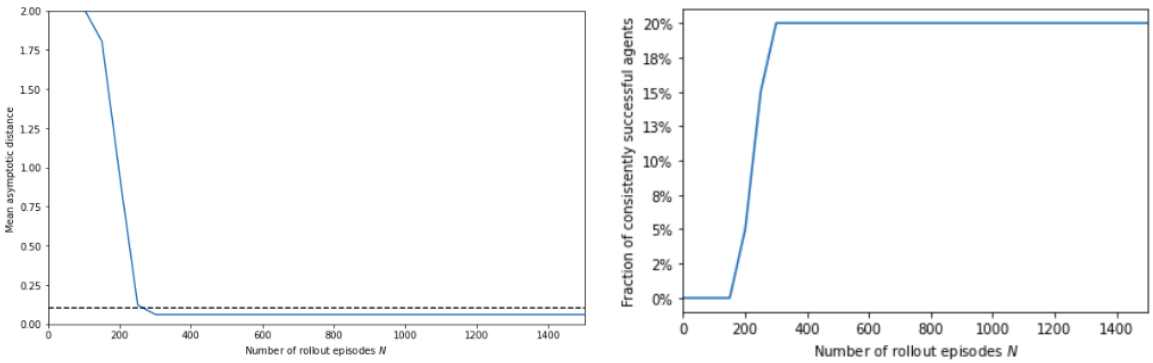

Figure 13: Evaluation of Zig-Zag Example through (Left) mean asymptotic distance to the goal as the rollouts increase and (Right) Fraction of consistently successful agents per rollout

5 Conclusion

This internship started with learning the basics of RL and testing it out on the open-sourced environments of OpenAI Gym [6]. The two environments that I used to get an idea of the normal Q-Learning method were Mountain-Car and the Cart-Pole. I tested Potential-Based Reward Shaping techniques on the mountain-car environment. After this, the next step was to implement and train a DQN, for which I used a blob game environment. After that, I moved-on to implement DDPG on the inverted pendulum environment.

After getting an idea of how RL works in a pre-defined environment, the next step was to set-up the RAI environment on an HPC cluster. This took some time, since the operating

system was different, but was quintessential in understanding the inner-workings of RAI, at least to the extent that allowed me to work on the environments later on. Once this set-up was completed, the experiments were performed on this environment, the details of which have been given in the previous section.

The results shown in the previous section demonstrate the effectiveness of the FVRS approach to inculcating plan-based priors into reward-shaping. The key idea that they exploit is the value stored in the intermediate states that the agent visits, in contrast to PBRS which looks at the initial and final configurations. This difference is analogous to the difference between the conservative and non-conservative forces in mechanics, where a force like the potential-energy of the spring depend only on the distances of two states w.r.t the mean as compared to a force like friction that depends on the mass and velocity of the object.

The idea of using 'closeness' or 'similarity' of the generated movement to a plan has its limitation in that the agent might go off if the plan is globally wrong, for example, in a scenario where it does not lead to the goal. However, if the plan leads to the goal, then subject to the constraints of the environment, the agent will figure out a policy under optimal control following a reward shaped by this plan. The true strength of this approach lies in the integration of a planner like KOMO [25] to generate this plan. This could lead to sample-effective learning using a plan generated in a simulation, that can be used in real-world learning.

A notable issue that came during these pertinent experiments was that of the agent saturating to a sub-optimal policy, and this was characterized by saturated q-values. In most of the cases, once an agent fell into this valley, it would never recover from it. There were 2 interesting observations during the experiments on this issue:

- Changing the width (w) of the shaping algorithm form 0.1 to 0.2 drastically reduced such saturation. While no conclusive analysis was done on this part by me, it was by manually tuning the parameter that this was discovered independently by me. As it turned out, the same result was obtained by my supervisor though a thorough analysis of the width values. One possible explanation could be that increasing the width allows the agent to inculcate more information and thus, improves its exploration to a certain extent, after which it degrades.
- There were cases in the zig-zag examples where the agents were able to get out of saturation. This could be because of the tight packing of the reward signals that are received from closely knitted paths that allow the agent to move away from saturation.

A more thorough analysis might be required.

5.1 Future Outlook

The results obtained in this internship open up the possibilities for more work in this domain. the obvious extension, as mentioned before, is the integration with KOMO that would allow an agent to exploit the planner. Another extension that I find extremely interesting is exploring the application of this approach in a Multi-Agent Setting, and thus, Multi-Agent Reinforcement Learning [30]. When we look into a scenario of agents co-operating or competing to achieve their common or different objectives, then the scenario becomes much more complicated since each agent now has to take the actions of the other agent into account while learning. Moreover, the plans now cannot be fixed for each rollout, and need to be recursively called based on the current configuration of the environment. Moreover, the central aspect of thew work - reward shaping - becomes the most interesting part as the scheme that would be used to assign these rewards would determine how the agents perform. Algorithms like MADDPG [14] propose training setups where the Actor networks are distributed amongst the agents and the Critic network is centralized to evaluate the quality fo the actions of all the agents.

During this internship, I took some steps in the direction of setting an adversarial scenario. Figure 14 shows one such setup where there are two end-effectors of different sizes that have the task of pushing the box to their respective goals and would have a dynamic plan associated with pushing this box at each point. Their rewards would be shaped by this plan at each step that they take.

The factors that add complexity to this problem are:

- The sizes of the end-effectors, which are different for both agents
- The starting position of the box w.r.t to each agent, which determines how the agents would attempt to reach the box
- The position of the goal w.r.t the position of each agent at any time point, which determines how each agent will try to reach the goal once it is in contact with the box

A variant of this scenario could be that the agent with the bigger end-effector first learns to move in a certain vicinity of the box and then it's goal is set to the second agent and it's target is to disrupt the motion of the other agent. In this case, the first agent needs to figure out how to push the box to a goal with through a path that prevents it from colliding with

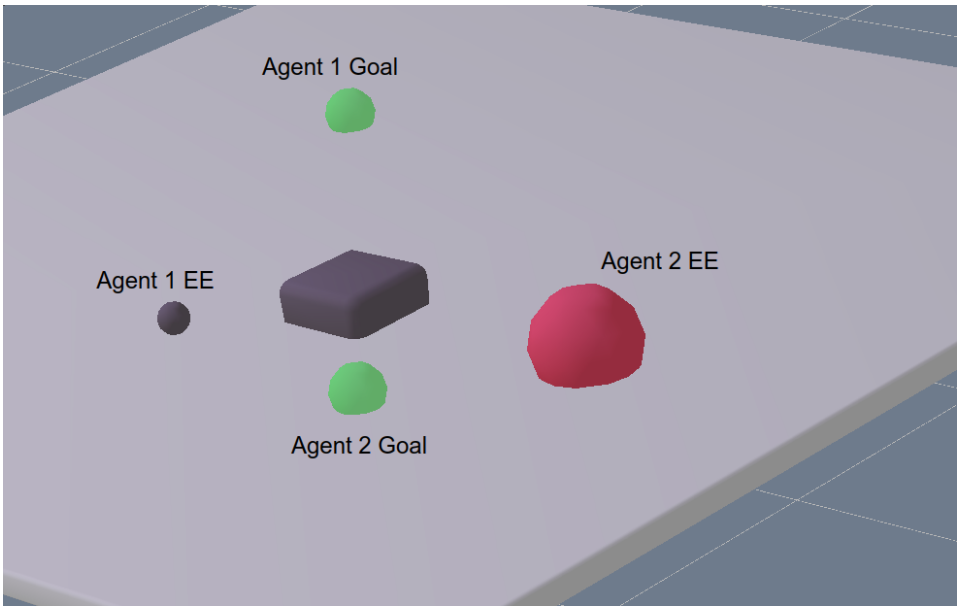

Figure 14: A 2 agent set-up where each agent has a different goal and they must compete against each other to push the box

the adversary.

This direction, along with KOMO integration, highly interest me since they open up the door to some novel work in the field of Reinforcement Learning. The exposure that this internship has given me to the field of RL has been instrumental in piquing my curiosity even further. I am convinced that I want to work further in this field and work on original research as part of my Master's Thesis. The directions that I would like to explore in this regard are exploration strategies and Multi-agent settings.

References

[1] A. Andrew. "Reinforcement Learning: An Introduction by Richard S. Sutton and Andrew G. Barto, Adaptive Computation and Machine Learning series, MIT Press (Bradford Book), Cambridge, Mass., 1998, xviii + 322 pp, ISBN 0-262-19398-1, (hardback, £31.95)". In: *Robotica* 17 (1999), pp. 229–235.

[2] Anonymous. "Plan-Based Asymptotically Equivalent Reward Shaping". In: *Submitted to International Conference on Learning Representations*. under double-blind review. 2021. URL: https://openreview.net/forum?id=w2Z2OwVNeK.

[3] *Atari 2600 Environment*. https://openai.com/blog/ingredients-for-robotics-research/.

[4] *Atari 2600 Environment*. https://www.endtoend.ai/envs/gym/atari/.

[5] RICHARD BELLMAN. "A Markovian Decision Process". In: *Journal of Mathematics and Mechanics* 6.5 (1957), pp. 679–684. ISSN: 00959057, 19435274. URL: http://www.jstor.org/stable/24900506.

[6] Greg Brockman et al. *OpenAI Gym*. cite arxiv:1606.01540. 2016. URL: http://arxiv.org/abs/1606.01540.

[7] Yuri Burda et al. "Exploration by Random Network Distillation". In: *ArXiv* abs/1810.12894 (2019).

[8] P.W.D. Charles. *Project Title*. https://github.com/charlespwd/project-title. 2013.

[9] *Exploration Strategies in Deep Reinforcement Learning*. https://lilianweng.github.io/lil-log/2020/06/07/exploration-strategies-in-deep-reinforcement-learning.html.

[10] Marek Grzes and Daniel Kudenko. "Plan-based reward shaping for reinforcement learning". In: vol. 31. Oct. 2008, pp. 10–22. DOI: 10.1109/IS.2008.4670492.

[11] *Introduction to Reinforcement Learning*. https://www.davidsilver.uk/wp-content/uploads/2020/03/intro_RL.pdf.

[12] Vijay Konda and John Tsitsiklis. "Actor-Critic Algorithms". In: *Society for Industrial and Applied Mathematics* 42 (Apr. 2001).

[13] T. Lillicrap et al. "Continuous control with deep reinforcement learning". In: *CoRR* abs/1509.02971 (2016).

[14] Ryan Lowe et al. "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments". In: *ArXiv* abs/1706.02275 (2017).

[15] M. Manhart. "Markov Processes". In: 2012.

[16] Martın Abadi et al. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[17] V. Mnih et al. "Human-level control through deep reinforcement learning". In: *Nature* 518 (2015), pp. 529–533.

[18] V. Mnih et al. "Playing Atari with Deep Reinforcement Learning". In: *ArXiv* abs/1312.5602 (2013).

[19] A. Ng, D. Harada, and S. Russell. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping". In: *ICML*. 1999.

[20] *NVIDIA PhysX*. https://developer.nvidia.com/gameworks-physx-overview.

[21] *Robotic AI (RAI) Environment*. https://github.com/MarcToussaint/rai.

[22] R. Sutton et al. "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *NIPS*. 1999.

[23] Pravati Swain, Purandar Bhaduri, and Sukumar Nandi. "Probabilistic model checking of IEEE 802.11 IBSS power save mode". In: *International Journal of Wireless and Mobile Computing* 7 (May 2014), pp. 465–474. DOI: 10.13140/2.1.1204.0640.

[24] *Tic-Tac-Toe Environment*. https://towardsdatascience.com/reinforcement-learning-implement-tictactoe-189582bea542.

[25] Marc Toussaint. "Newton methods for k-order Markov Constrained Motion Problems". In: *ArXiv* abs/1407.0414 (2014).

[26] A. Trott et al. "Keeping Your Distance: Solving Sparse Reward Tasks Using Self-Balancing Shaped Rewards". In: *ArXiv* abs/1911.01417 (2019).

[27] Chris Watkins and P. Dayan. "Q-learning". In: *Machine Learning* 8 (2004), pp. 279–292.

[28] Eric Wiewiora. "Reward Shaping". In: *Encyclopedia of Machine Learning*. Ed. by Claude Sammut and Geoffrey I. Webb. Boston, MA: Springer US, 2010, pp. 863–865. ISBN: 978-0-387-30164-8. DOI: 10.1007/978-0-387-30164-8_731. URL: https://doi.org/10.1007/978-0-387-30164-8_731.

[29] M. Wooldridge and N. Jennings. "Intelligent agents: theory and practice". In: *Knowl. Eng. Rev.* 10 (1995), pp. 115–152.

[30] K. Zhang, Zhuoran Yang, and T. Başar. "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms". In: *ArXiv* abs/1911.10635 (2019).